

Taller 3 - Unidad Un Poco Aritmético-Lógica

Sistemas Digitales

Segundo Cuatrimestre 2026

1. Enunciado

Este taller es una **adenda** del Taller 2: se arman AND/OR de 4 bits, se releen el sumador y el restador, y se conecta todo en una UUPAL de 4 bits para jugarla en HDL Studio. Sirve de repaso antes de máquinas de estados.

Usar el paquete inicial del Taller 3 (devcontainer, esqueletos `.sv` y testbenches) de la página de la materia.

Importante: Este taller **no se entrega**. No hay que subir nada al campus.

2. Cómo trabajar

- a) No cambiar nombres de archivos, módulos ni puertos.
- b) No editar los testbenches.
- c) Instanciar los módulos de ejercicios anteriores. No reescribir el sumador ni el restador a mano.
- d) Sintaxis: la guía incremental `sintaxis.typ`. No usar `for`, `generate` ni `enum`.
- e) Resolver en orden. `make sim` copia a la carpeta los `.sv` anteriores: no editar esas copias.
- f) En cada ejercicio: correr los tests (`make sim`) y sintetizar el circuito en HDL Studio. Corroborar **a mano** que funciona (cambiar entradas, ver salidas). Agregar al circuito todos los `.sv` de la carpeta **salvo** el `_tb`.

3. Ejercicios

3.1. Ejercicio 1: AND de 4 bits

En ejercicios/ej1, completar `and_4b.sv` (`compuerta_and_4b`). Cada bit de `result` vale 1 solo si los dos bits de `a` y `b` valen 1.

3.2. Ejercicio 2: OR de 4 bits

En ejercicios/ej2, completar `or_4b.sv` (`compuerta_or_4b`). Cada bit de `result` vale 1 si al menos uno de los bits de `a` y `b` vale 1.

3.3. Ejercicio 3: Leer el sumador de 4 bits

En ejercicios/ej3 los tres archivos **ya están resueltos**. Leer `sumador_simple`, `sumador_completo` y `sumador_4b`. Seguir el acarreo de `cin` a `cout`. Correr los tests y sintetizar. No hay que modificar nada.

3.4. Ejercicio 4: Leer el restador de 4 bits

Igual que el 3, con `restador_simple`, `restador_completo` y `restador_4b`. Seguir el préstamo de `bin` a `bout`. Correr los tests y sintetizar. No modificar.

3.5. Ejercicio 5: UUPAL

En ejercicios/ej5, completar `uupal.sv`. Reutilizar `AND`, `OR`, `sumador_4b`, `restador_4b` y `registro_4b`.

La máquina tiene:

- banco R0–R3 (cada uno con su `we`);
- operandos de ALU `operand_a` y `operand_b`;
- `AND`, `OR`, suma y resta **en paralelo**;
- un mux `op` que elige el resultado.

Señal	Qué hace
<code>we0..we3</code>	escribe ese registro con el bus de escritura
<code>force_en</code>	1 → el bus es <code>force_in</code> ; 0 → el bus es <code>result</code>
<code>src_a</code> / <code>src_b</code>	de qué registro lee cada operando
<code>load_op_a</code> / <code>load_op_b</code>	captura ese bus en el flanco (<code>ALU_A</code> / <code>ALU_B</code>)
<code>op</code>	elige la operación de la ALU

Código	<code>src_a/src_b</code>	Código	<code>op</code>
00	R0	00	AND
01	R1	01	OR
10	R2	10	ADD
11	R3	11	SUB

Suma y resta módulo 16: no guardar `cout` ni `bout`. `cin` y `bin` van a 0.

Completar el módulo, pasar `make sim` y sintetizar. Después, **a mano** en el circuito, hay que ejecutar esta receta. Cada fila es lo que vale **entre un flanco y el siguiente** (las señales que tienen que estar activas cuando llega el clock):

- cargar `A` (hex, 10) en `r0`;
- cargar 1 en `r1`;

- c) cargar **r0** en ALU_A (**operand_a**);
- d) cargar **r1** en ALU_B (**operand_b**);
- e) hacer la **suma** y guardar el resultado en **r3**.

Por ejemplo

Ciclo	Qué	Señales (entre flancos)
1	cargar r0 en A	load_op_a , src_a en 00
2		
3		
4		
5		

Al terminar, **r3** tiene que valer **A + 1** (**B** hex, 0110 en binario).